# Technical Documentation

Wilson Tennis — Performance Series Landing Page

---

## Table of Contents


1. Architecture Overview
2. Technologies & Tools
3. How It Works
    - Application Startup
    - Loading Screen
    - 3D Racket Scene
    - Scroll Animation System
    - State Bridge (scrollStore / loadStore)
4. Component Breakdown
5. Animation Details
6. Design System
7. Responsive Behavior


---

## Architecture Overview

The app is built on a **three-layer architecture** that separates concerns between the DOM, application state, and the WebGL canvas:

```
┌──────────────────────────────────────────┐
│           DOM Layer (React)              │
│  Navbar, Hero, Features, Technology,    │
│  Collection, Ticker, Footer              │
└──────────────────┬───────────────────────┘
                   │ reads/writes
┌──────────────────▼───────────────────────┐
│         State Bridge (lib/)              │
│  scrollStore.js  ↔  loadStore.js         │
│  (mutable progress float)  (callbacks)   │
└──────────────────┬───────────────────────┘
                   │ reads every frame
┌──────────────────▼───────────────────────┐
│        WebGL Layer (R3F Canvas)          │
│  RacketScene.jsx — fixed position,       │
│  runs its own render loop via useFrame   │
└──────────────────────────────────────────┘
```

React's state system cannot bridge into R3F's `useFrame` loop without causing re-renders, so a plain mutable object (`scrollStore`) is used instead. `ScrollAnimationManager` writes to it via GSAP ScrollTrigger callbacks; `RacketScene` reads from it each frame to drive the 3D model.

## Technologies & Tools

### Core

| Tool | Version | Why it was used |
|---|---|---|
| **React** | 19.2 | Component model for building the UI in a declarative way |
| **Vite** | 8.0 | Fast dev server with HMR; minimal config for React + R3F projects |

### 3D Rendering

| Tool | Version | Why it was used |
|---|---|---|
| **Three.js** | 0.183 | Industry-standard WebGL library for 3D in the browser |
| **@react-three/fiber** | 9.5 | React renderer for Three.js — lets you write 3D scenes as JSX components |
| **@react-three/drei** | 10.7 | Utility components: `useGLTF` for model loading, `Environment` for HDR lighting |

### Animation

| Tool | Version | Why it was used |
|---|---|---|
| **GSAP** | 3.14 | Precise timeline control, easing curves, and the ScrollTrigger plugin |
| **@gsap/react** | 2.1 | `useGSAP` hook that correctly scopes and cleans up GSAP contexts in React |
| **gsap/ScrollTrigger** | (plugin) | Maps DOM scroll position to animation progress; used for per-section triggers |

### Styling

| Tool | Version | Why it was used |
|---|---|---|
| **Tailwind CSS** | 4.2 | Utility-first CSS; v4's native CSS variable support drives the custom theme |
| **@tailwindcss/vite** | 4.2 | Vite plugin for Tailwind v4's new CSS-first config approach |

### Dev Tooling

| Tool | Version | Purpose |
|---|---|---|
| ESLint | 9.39 | Linting with `eslint-plugin-react-hooks` and `eslint-plugin-react-refresh` |
| @vitejs/plugin-react | 6.0 | JSX transform and Fast Refresh |

# How It Works

## Application Startup

`main.jsx` registers `ScrollTrigger` with GSAP before the app mounts:

```
gsap.registerPlugin(ScrollTrigger);
```

`App.jsx` renders all components and manages two pieces of React state:

- `modelReady` — set to `true` when the GLTF model finishes loading
- `canPlay` — set to `true` after the loading screen exit animation completes, which gates the Hero entrance animation

## Loading Screen

`LoadingScreen.jsx` runs a GSAP timeline on mount:

1. The green progress bar animates from `0%` → `80%` over 2.5 seconds using `power1.out` easing. It intentionally stops at 80% to wait for the real model load signal.
2. When `modelReady` becomes `true` (passed as a prop), a second GSAP context fires: the bar snaps to `100%`, the loading content fades up and out, and the entire overlay slides off the top of the viewport with `power3.inOut` easing.
3. An `onComplete` callback calls `onFinish()` which sets `canPlay = true` in `App.jsx`, triggering the Hero entrance animation.

## 3D Racket Scene

`RacketScene.jsx` is an R3F `<Canvas>` that is **fixed to the viewport** via a CSS wrapper, so it stays visible across all scroll sections. The canvas itself has `pointer-events: none` so it doesn't block DOM interactions.

Inside the canvas:

- `useGLTF` loads the Wilson Blade GLTF model from `/public/models/`
- `loadStore.onLoaded?.()` is called once loading completes, notifying `App.jsx`
- The scene has ambient lighting (0.4 intensity) + two directional lights + an HDR `Environment` preset set to `"city"`
- The model is responsive: the scale multiplier is `0.65` on mobile (< 768px), `0.8` on tablet, and `1.0` on desktop

Every frame, `useFrame` reads `scrollStore.progress` and lerps the racket's `position`, `rotation`, and `scale` toward the target keyframe values using `THREE.MathUtils.lerp` with a smoothing factor of `0.1`. This creates the silky eased motion as the user scrolls.

## Scroll Animation System

`ScrollAnimationManager.jsx` creates five `ScrollTrigger` instances — one per page section (hero, features, technology, collection, footer). Each trigger maps its `0 → 1` progress to a section offset:

| Section | Progress Range |
|---|---|
| Hero | 0.0 → 1.0 |
| Features | 1.0 → 2.0 |
| Technology | 2.0 → 3.0 |
| Collection / Footer | 3.0 → 4.0 |

The resulting float is written directly to `scrollStore.progress`. `RacketScene` reads it each frame to select and interpolate between the correct pair of keyframes from `racketKeyframes.js`.

### Racket Keyframes

`racketKeyframes.js` defines 5 keyframe objects, each with `position`, `rotation`, and `scale`:

| Keyframe | Position | Scale | Description |
|---|---|---|---|
| 0 — Hero | `[2, -1, -2]` | 1.0 | Large, right-side showcase |
| 1 — Features | `[0, 0, 0]` | 0.5 | Centered, small for diagram overlay |
| 2 — Technology | `[-1.2, 0, -1]` | 0.8 | Left-back, angled |
| 3 — Collection | `[2, 0.5, 0]` | 0.9 | Right, tilted |
| 4 — Footer | `[3, -1, -2]` | 0.6 | Far right, exiting |

### State Bridge

Two plain JavaScript modules in `src/lib/` act as cross-context communication channels:

**`scrollStore.js`**

```
export const scrollStore = { progress: 0 };
```

A single mutable object. `ScrollAnimationManager` writes to `scrollStore.progress`; `RacketScene`'s `useFrame` reads it. No React state, no event emitter — just a shared reference, which is safe because R3F's render loop runs outside React's reconciler.

**`loadStore.js`**

```
export const loadStore = { onLoaded: null };
```

Holds a callback reference. `App.jsx` sets `loadStore.onLoaded = () => setModelReady(true)`. `RacketScene` calls it after `useGLTF` resolves. This avoids prop drilling the callback down through the R3F canvas boundary.

---

## Component Breakdown

### `Navbar.jsx`

Fixed to the top with a blurred backdrop (`backdrop-blur-md`). Contains the Wilson logo (stylized "W" in accent green), navigation links, and a "Shop Now" CTA button. Links use smooth scroll via anchor `href` attributes.

### `Hero.jsx`

Full-viewport first section. Uses `useGSAP` to build a timeline that animates:

- Geometric decorations (rotated borders, dot grid, diagonal lines, triangles) — scale from `0.4 → 1` with stagger
- Label tag — opacity + y translate
- Two headline lines ("Precision", "Under Tension") — opacity + y translate with `0.15s` stagger
- Subtitle paragraph and two CTA buttons — opacity + y translate

The timeline only plays when `canPlay` (prop from `App.jsx`) becomes `true`, ensuring it doesn't run before the loading screen exits.

A `animate-bounce` scroll indicator sits at the bottom of the section.

### `Features.jsx`

Shows four technical spec cards — String Bed, Frame, Throat, Grip — positioned around the 3D racket model.

On desktop: cards are absolutely positioned at the corners/sides with SVG `<line>` elements connecting them visually to the racket. The SVG animation uses `strokeDasharray` / `strokeDashoffset` to draw the lines progressively, combined with a `feGaussianBlur` glow filter. Endpoint dots scale in with `back.out(2)` for a pop effect.

On mobile: the cards collapse into a 2×2 grid below the racket.

All elements trigger on scroll using `ScrollTrigger` with `toggleActions: "play reverse play reverse"`.

### `Ticker.jsx`

Two rows of infinitely scrolling marquee text using GSAP:

- Row 1 (accent green background): scrolls left — brand keywords and taglines
- Row 2 (dark background): scrolls right — performance keywords

Each row contains two copies of the text side by side. GSAP animates `xPercent` from `-50` to `-100` (row 1) or `-50` to `0` (row 2) over 50 seconds with `repeat: -1` and `ease: "none"` for constant speed. The section uses a CSS `skewY` transform for a dynamic perspective effect.

### `Technology.jsx`

A four-column grid of material spec cards (Carbon Fiber, String Bed, Grip System, Vibration Dampener), each showing an icon, a headline stat, and a description paragraph. Below the grid is a featured callout panel for "Carbon Fiber Layup Technology" with four detailed stat boxes.

All elements are scroll-triggered with staggered entrance animations (`y: 30 → 0`, `opacity: 0 → 1`).

### `Collection.jsx`

A grid of four racket product cards:

| Racket | Weight | Balance | Tagline |
|---|---|---|---|
| Shift 99 | 316g | 32cm | Spin Meets Power |
| Pro Staff One | 331g | 30.5cm | Precision in every swing |
| Ultra 100 | 300g | 33cm | Power without compromise |
| Blade 98 | 305g | 33cm | Built for the Bold |

Each card splits into two panels:

- **Left** — product image with a hover effect that swaps to an alternate angle using CSS `opacity` transitions
- **Right** — slanted accent panel using `polygon clip-path`, racket name, tagline, and specs

On scroll, even-indexed cards enter from the top (`y: -80`) and odd-indexed from the bottom (`y: +80`), creating a converging entrance effect.

### `LoadingScreen.jsx`

Overlay with a Wilson logo, animated progress bar, and loading label. See Loading Screen above for full behavior.

### `ScrollAnimationManager.jsx`

A render-less component (returns `null`) that creates and manages all `ScrollTrigger` instances. Cleans them up on unmount.

### `Footer.jsx`

Simple footer row with navigation links and the project disclaimer: "*Personal project — not affiliated with Wilson Sporting Goods.*"

---

## Animation Details

### GSAP Timeline (Hero)

```
t=0.0s  geometric shapes    scale(0.4 → 1)    stagger 0.12s
t=0.3s  label               opacity+y         0.5s
t=0.5s  headline line 1     opacity+y         0.6s
t=0.6s  headline line 2     opacity+y         0.6s   (stagger 0.15s)
t=0.8s  subtitle            opacity+y         0.6s
t=0.9s  buttons             opacity+y         0.5s   (stagger 0.1s)
```

### GSAP ScrollTrigger (per section)

- `start: "top 80%"` — triggers when the section top is 80% down the viewport
- `end: "bottom 20%"` — ends when the section bottom is 20% from the top
- `toggleActions: "play reverse play reverse"` — animates in and out as the user scrolls both ways

### 3D Interpolation (per frame)

```
model.position.x = lerp(model.position.x, target.position[0], 0.1)
model.position.y = lerp(model.position.y, target.position[1], 0.1)
model.position.z = lerp(model.position.z, target.position[2], 0.1)
// same for rotation.x/y/z and scale
```

The `0.1` factor gives ~10% progress toward the target per frame at 60fps, resulting in a smooth exponential ease-out.

---

## Design System

All design tokens are defined as CSS variables in `src/index.css` and exposed to Tailwind v4 via `@theme`:

| Token | Value | Usage |
|---|---|---|
| `--color-bg` | `#1A1918` | Page background (dark charcoal) |
| `--color-accent` | `#73ce44` | Wilson green — buttons, highlights, borders |
| `--color-secondary` | `#747d90` | Muted blue-gray — subtext, icons |
| `--color-text` | `#FFFFFF` | Primary text |
| `--color-muted` | `#B0B3B8` | Secondary text, descriptions |
| `--color-glass` | `rgba(26,25,24,0.6)` | Frosted glass card backgrounds |
| `--color-glass-border` | `rgba(115,206,68,0.15)` | Subtle green card borders |

### Typography

| Font | Usage |
|---|---|
| Archivo Black | Headings — bold, all-caps |
| Inter | Body text, UI labels |
| Roboto Mono | Stats, specs, technical labels |

---

## Responsive Behavior

| Breakpoint | Layout Changes |
|---|---|
| Mobile (< 640px) | Single-column layout; Features grid hidden, SVG connectors hidden; 3D model scaled to 65% |
| Tablet (640–1024px) | Two-column grid for Collection; 3D model at 80% scale |
| Desktop (> 1024px) | Full layout with SVG feature connectors; 3D model at full scale; Navbar links visible |

The 3D racket model scale is computed once on mount and on window resize using a simple breakpoint check, then applied as a uniform scale multiplier on top of the per-keyframe base scale.